How-To: ragent Configuration (`ragent.json`)

ragent is configured through a single JSON file called `ragent.json`. This document is the complete reference for every section of that file: what it controls, the schema, the defaults, and worked examples.

If you only want to get running quickly, read the **Quick Start** section in the project root `README.md` and the **Tutorial** in `docs/howtos/tutorial.md`. This document is the deep reference.

Table of Contents

1. Overview
2. File Discovery and Loading Order
   - Auto-creation
   - Load cache and save invalidation
3. Merge Semantics
4. CLI and Environment Overrides
5. Config Error Reporting
6. Backup and Restore
7. Section Reference
   - 7.1 `username`
   - 7.2 `defaultAgent`
   - 7.3 `provider`
   - 7.4 `agent`
   - 7.5 `permission`
   - 7.6 `command`
   - 7.7 `mcp`
   - 7.8 `instructions`
   - 7.9 `skill_dirs`
   - 7.10 `experimental`
   - 7.11 `hooks`
   - 7.12 `bash`
   - 7.13 `dirs`
   - 7.14 `code_index`
   - 7.15 `stream`
   - 7.16 `memory`
   - 7.17 `compaction`
   - 7.18 `gitlab`
   - 7.19 `yolo`
   - 7.20 `edit_log`
   - 7.21 `prices`
   - 7.22 `telemetry`
   - 7.23 `agent_perf`
   - 7.24 Search API keys
   - 7.25 `sdd`
   - 7.26 `trigger`
   - 7.27 `piegap`
   - 7.28 `research`
   - 7.29 `loop`

    - 7.30 `activity_log`
    - 7.31 `plugins`
    - 7.32 `input_queue_capacity`
    - 7.33 `connectors`
    - 7.34 `execution_backend` and `backends`
    - 7.35 `acp`
    - 7.36 `i18n`
    - 7.37 `security_analyzer`
    - 7.38 `openai`
    - 7.39 `automation`
    - 7.40 `tools`
    - 7.41 `agents`
    - 7.42 `storage`
8. Full Example File
9. Common Recipes
10. Related Documents

---

1. Overview

All ragent runtime behaviour is driven by a single layered JSON configuration:

- **Provider connections** (API keys, base URLs, thinking settings, model catalogues)
- **Agent defaults and overrides** (system prompts, max steps, skills)
- **Permissions** (allow / deny / ask rules, bash and directory allow/deny lists)
- **Memory, compaction, and retrieval** settings
- **Telemetry / OpenTelemetry export**
- **External integrations** (GitLab)
- **Feature flags** (experimental, SDD, pie-gap, triggers, research)

The config is loaded once at startup by `ragent-config::Config::load`, merged across layers, and then held in memory for the lifetime of the process. Several subsystems (YOLO mode, edit logging, bash/directory lists) also read runtime atomic flags that are synced from the config at startup.

Design principles

Principle	Meaning
Layered merge	Global config provides defaults; project config overrides; env vars override both.
Explicit-over-default	Many fields track whether the user *explicitly* set them so a serialise round-trip does not lose a toggle.
Sensible defaults	Every section has `Default` implementations so an empty `{}` is valid.
No secrets in config by force	API keys come from env vars or `env:VAR_NAME` indirection; the config file can reference them but does not have to store them.

Principle	Meaning
Atomic persistence	Writes use temp-file-then-rename so a crash never leaves a partial config.

---

2. File Discovery and Loading Order

`Config::load` resolves configuration in a strict precedence order. Each layer is merged on top of the previous one, with later layers overriding earlier ones for explicitly-set fields.

```
compiled defaults
  -> global file:   <config_dir>/ragent/ragent.json
  -> project file:  ./.ragent/ragent.json
  -> env file:      $RAGENT_CONFIG  (path to a JSON file)
  -> env inline:    $RAGENT_CONFIG_CONTENT  (raw JSON string)
```

Platform config directory

The global config directory is resolved by the `dirs` crate:

Platform	Path
Linux	`$XDG_CONFIG_HOME/ragent/ragent.json` (default `~/.config/ragent/ragent.json`)
macOS	`~/Library/Application Support/ragent/ragent.json`
Windows	`%APPDATA%\ragent\ragent.json`

The helper `Config::global_config_dir()` returns `<config_dir>/ragent` and `Config::global_config_path()` returns the full `ragent.json` path.

Project config

The project config lives at `./.ragent/ragent.json` relative to the current working directory. This is the file created automatically on first run when neither a global nor project config exists.

JSONC note

The crate-level documentation mentions `ragent.jsonc` as a supported filename, and JSONC-style comments appear in documentation examples (`//` line comments). However, the parser uses `serde_json::from_str`, which is strict JSON. Inline `//` comments are **not** supported in the actual config file. Use a `.json` extension and plain JSON. The `jsonc` examples in code doc-comments are for illustration only.

Auto-creation

If no config file is found at either the global or project path, `Config::load` creates a `.ragent/` directory and writes a default `ragent.json` there so the user has a starting point.

Load cache and save invalidation

`Config::load()` keeps a process-level cache of the resolved configuration, keyed on the current working directory plus the `(path, mtime, size)` tuple of every candidate config file that existed at load time. Two rules keep the cache coherent (the M-025 discipline):

- The mtime snapshot is taken **after** `load_uncached()` runs, so a first-time load that auto-creates `.ragent/ragent.json` includes the new file in the cache key.
- Every successful write through `Config::save()` or `Config::save_to_source()` calls `Config::invalidate_load_cache()`, so the next `load()` re-reads from disk instead of serving the pre-save config.

Together these guarantee that TUI saves (`/codeindex off`, `/tools`, …) are immediately visible to subsequent loads in the same process — including the case where the save created a config file that was absent from the cached mtime snapshot (for example, a fresh environment with no global config, as on CI).

---

3. Merge Semantics

`Config::merge(base, overlay)` combines two configs. Different sections use different merge strategies:

Project overlays are untrusted (SEC-001/SEC-002/SEC-005). The project file (`./.ragent/ragent.json`) is repository content, so `Config::merge_project` strips the privileged keys before the merge and logs each rejected key at `warn!`. A project file may **not** set: `yolo`, permission allow-rules, the `bash`/`dirs` lists, `telemetry`, `hooks`, `openai`, `mcp` (a stdio server spawns a subprocess with no permission prompt), `acp` (an ACP agent spawns verbatim, and the section controls the editor-facing server), or any provider's `api` block (which redirects the provider credential and prompt stream). Put those in the user-global `~/.config/ragent/ragent.json`. The rest of a project provider entry (`thinking`, `env`, `models`) still merges.

Section	Strategy
`username`, API keys, `gitlab`	Overlay overrides base when `Some`.
`defaultAgent`	Overlay overrides when explicitly set or different from default.
`provider`	Deep merge per provider key: provider-level `api`, `env`, `thinking`, and `options` are replaced when the overlay specifies them, while `models` entries are merged per-model-id so a partial overlay (e.g. `provider.openrouter.models."anthropic/claude-sonnet-4"`) does not discard lower-layer fields such as `name` or `capabilities`. Model-level `thinking` overrides provider-level `thinking`.
`agent`, `command`, `mcp`	Per-key overlay replaces base entry.
`permission`, `instructions`, `skill_dirs`, `hooks`	Append (union).
`bash.allowlist`, `bash.denylist`	Union (deduplicated).
`prices`	Per-model last-wins (overlay entry replaces base entry for the same model id).

Section	Strategy
`code_index`	Overlay wins **only** for explicitly-set fields. The `specified` flags are propagated so the value survives a reload.
`compaction`	Overlay replaces base entirely.
`yolo`, `edit_log`	OR semantics: if the overlay sets `true`, it wins.
`sdd`, `piegap`	OR semantics: a flag enabled in either layer stays enabled.
`research.open_access_recovery`, `research.exclude_academic_engines`	OR semantics. `contact_email`, `oa_min_full_text_chars`, `max_concepts`, and `max_findings`: overlay overrides when present (and different from the default).
`telemetry`	If overlay enables telemetry, the whole overlay `otel` block replaces the base. Otherwise maps (`resource_attributes`, `metrics`) are unioned.
`experimental`	`open_telemetry` and `parallel_tool_calls`: OR semantics. Other fields: overlay defaults.
`plugins`	Overlay wins wholesale when the `plugins` section is present, so a project-level block is not discarded by an absent user-global block.
`connectors`	Overlay wins wholesale when the `connectors` section is present, so a project-level block is not discarded by an absent user-global block.
`input_queue_capacity`	Overlay wins when explicitly set, so a project-level override is not discarded by an absent user-global value.
`tools`	`aliases` entries union **per spelling** (the overlay wins a collision, so a project can add one spelling without discarding the user-global table); `expose_alias_names` and `wire_trace` use OR semantics. The one exception is an overlay that writes `aliases: false`, which switches resolution off and drops the inherited entries - `false` is the only way an author can say “no table”, and an unconditional union would make a user-global `true` impossible to switch off. An empty object `{}` is *not* that switch.

The `config_paths` field records every file that contributed to the final merged config, in load order.

---

4. CLI and Environment Overrides

`--config <PATH>` CLI flag

```
ragent --config /path/to/my-config.json run "hello"
```

When `--config` is supplied, the file at that path is loaded directly and bypasses the normal discovery order. Parse errors for this path include the file path, line, column, the problematic source line, and a caret

marker.

Environment variables

Variable	Purpose
`RAGENT_CONFIG`	Path to a JSON config file, merged on top of project config.
`RAGENT_CONFIG_CONTENT`	Raw JSON string, merged last (highest precedence). Parse errors show the inline content.
Provider API keys	See section 7.3. Env vars always take precedence over config-file values.
`TAVILY_API_KEY`	Overrides `tavily_api_key`.
`OPENALEX_EMAIL`	Overrides `openalex_email`.
`EXA_API_KEY`	Overrides `exa_api_key`.
`SERPER_API_KEY`	Overrides `serper_api_key`.
`GITLAB_TOKEN`, `GITLAB_URL`, `GITLAB_USERNAME`	Override `gitlab` fields (highest priority).

Other CLI flags that affect config

Flag	Effect
`--model <provider/model>`	Overrides the resolved model.
`--agent <name>`	Overrides `defaultAgent`.
`--yes` / `--no-prompt`	Auto-approves all permissions (like YOLO but does not persist).

---

5. Config Error Reporting

When a config file fails to parse, ragent produces an actionable diagnostic:

```
Failed to parse config file '.ragent/ragent.json':
Error at line 12, column 8:

Problematic line:
    "defaultAgent: "coder"
       ^
Parse error: expected `:` at line 12 column 8
```

This includes:

- The file path (or `RAGENT_CONFIG_CONTENT` for the inline env var).
- The line and column from the serde error.
- A separator line.
- The exact source line that caused the failure.
- A caret (^) positioned under the error column.
- The underlying serde message.

The same format is used for `--config`, `RAGENT_CONFIG`, and `RAGENT_CONFIG_CONTENT` errors.

---

6. Backup and Restore

`Config::backup_global_config()` snapshots the current global `ragent.json` into a timestamped file inside `<config_dir>/ragent/saves/`. The backup name uses `YYYY-MM-DD.HH-MM-SS` (hyphens in the time portion for Windows NTFS compatibility). If multiple saves happen in the same second, a counter is appended. The write is atomic (temp-file-then-rename).

`Config::restore_global_config()` restores a backup over the global `ragent.json`. It validates that:

- The backup file exists and is a regular file.
- The resolved destination is exactly `<config_dir>/ragent.json`.
- The backup content is valid JSON.

The restore is also atomic.

These are used by the `/config backup` and `/config restore` slash commands.

---

7. Section Reference

7.1 `username`

Field	Type	Default	Description
`username`	`Option<String>`	`None`	Display name of the user, used in prompts and logs.

```jsonc
{ "username": "alice" }
```

7.2 `defaultAgent`

Field	Type	Default	Description
`defaultAgent` (alias `default_agent`)	`String`	`"general"`	Name of the agent used when none is specified via `--agent`.

```jsonc
{ "defaultAgent": "coder" }
```

The loader tracks whether the user explicitly set this field (`specified_default_agent`) so merge operations do not accidentally overwrite an explicit project-level choice with a global default.

7.3 `provider`

The `provider` section is the most important part of the config. It is a map keyed by provider id, where each entry configures one LLM provider.

```jsonc
{
  "provider": {
    "anthropic": {
      "env": ["ANTHROPIC_API_KEY"],
      "thinking": { "enabled": true, "level": "low" },
      "models": {
        "claude-sonnet-4-20250514": {
          "thinking": { "enabled": true, "level": "high", "budget_tokens": 16000 }
        }
      }
    },
    "generic_openai": {
      "env": ["GENERIC_OPENAI_API_KEY"],
      "api": { "base_url": "http://127.0.0.1:8080" }
    }
  }
}
```

`ProviderConfig` schema

Field	Type	Default	Description
`env`	`Vec<String>`	`[]`	Environment variable names required by this provider. ragent checks these are set before model discovery.
`api`	`Option<ApiConfig>`	`None`	Optional API endpoint and header overrides.
`thinking`	`Option<ThinkingConfig>`	`None`	Default thinking/reasoning config for all models under this provider. Overridden by per-model `thinking`.
`models`	`HashMap<String, ModelConfig>`	`{}`	Model definitions keyed by model id.
`options`	`HashMap<String, Value>`	`{}`	Arbitrary provider-specific options.

`ApiConfig` schema

Field	Type	Default	Description
`base_url`	`Option<String>`	`None`	Base URL for API requests, overriding the provider default.
`headers`	`HashMap<String, String>`	`{}`	Extra HTTP headers sent with every request.

`ModelConfig` schema

Field	Type	Default	Description
`name`	`Option<String>`	`None`	Human-readable display name.
`cost`	`Option<Cost>`	`None`	Per-token pricing (USD per million tokens).
`capabilities`	`Option<Capabilities>`	`None`	Feature flags for this model.
`thinking`	`Option<ThinkingConfig>`	`None`	Per-model thinking config; overrides provider-level default.

`Cost` schema

Field	Type	Description
`input`	`f64`	Cost per million input tokens (USD).
`output`	`f64`	Cost per million output tokens (USD).

`Capabilities` schema

Field	Type	Default	Description
`reasoning`	`bool`	`false`	Supports chain-of-thought reasoning.
`streaming`	`bool`	`true`	Supports streaming responses.
`vision`	`bool`	`false`	Can process image inputs.
`tool_use`	`bool`	`true`	Supports tool/function calling.
`thinking_levels`	`Vec<ThinkingLevel>`	`[]`	Which thinking/reasoning levels this model supports.

`ThinkingConfig` schema Defined in `ragent-types`, this is shared across providers.

Field	Type	Default	Description
`enabled`	`bool`	`true`	Whether thinking is enabled. When `false`, `level` is ignored.
`level`	`ThinkingLevel`	`auto`	Thinking depth: `auto`, `off`, `low`, `medium`, `high`.
`budget_tokens`	`Option<u32>`	`None`	Max tokens for thinking (Anthropic `budget_tokens`). `None` = provider default.
`display`	`Option<ThinkingDisplay>`	`None`	How thinking content is surfaced: `full`, `summarized`, `omitted`.

`ThinkingLevel` maps to provider-native parameters:

Provider	Native parameter
Anthropic	`thinking.type` + `effort` / `budget_tokens`
OpenAI / Copilot	`reasoning_effort` (`low`, `medium`, `high`, `none`)
Gemini	`thinkingConfig.thinkingLevel` (`minimal`, `low`, `medium`, `high`, `auto`)
Ollama	`think` boolean

Supported provider ids Each provider is implemented in `crates/ragent-llm/src/providers/`. The provider id (from `fn id() -> &'static str`) is the key in the `provider` map.

Provider id	Env var(s)	Notes
`anthropic`	`ANTHROPIC_API_KEY`	Claude models. Supports thinking/budget_tokens.
`openai`	`OPENAI_API_KEY`	GPT models. Supports reasoning_effort.
`openai_responses`	`OPENAI_API_KEY`	OpenAI Responses API variant.
`gemini`	`GOOGLE_API_KEY` or `GEMINI_API_KEY`	Google Gemini. Supports thinkingConfig.
`ollama`	`OLLAMA_API_KEY` (optional for local)	Local Ollama. `think` boolean for thinking.
`ollama_cloud`	`OLLAMA_API_KEY`	Ollama Cloud variant.
`huggingface`	`HF_TOKEN` or `HUGGING_FACE_HUB_TOKEN`	Hugging Face Inference.
`copilot`	`GITHUB_COPILOT_TOKEN` or `GITHUB_TOKEN` (via `gh auth`)	GitHub Copilot.
`generic_openai`	`GENERIC_OPENAI_API_KEY` or `OPENAI_API_KEY`; `GENERIC_OPENAI_API_BASE` for base URL	Any OpenAI-compatible endpoint.
`azure_foundry`	`AZURE_AI_FOUNDRY_API_KEY`; `AZURE_AI_FOUNDRY_BASE` for base URL	Azure AI Foundry.
`azure_resource`	(from `azureresources.json`)	Azure Resource (File) provider. Reads model definitions from a separate `azureresources.json` file.

Provider id	Env var(s)	Notes
bedrock	AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, AWS_REGION / AWS_BEDROCK_REGION, optional AWS_SESSION_TOKEN	Amazon Bedrock. Uses AWS SigV4.
xai	XAI_API_KEY; XAI_API_BASE for base URL	xAI (Grok).
router	(aggregates other providers)	Model Router virtual provider. Classifies prompts into tiers and routes to the best model.

Model Router configuration The router is configured under `provider.router` and has its own `RouterConfig`:

```
{
  "provider": {
    "router": {
      "enabled": true,
      "tiers": {
        "SIMPLE":    { "models": [{ "provider": "ollama", "model": "llama3.2" }] },
        "MEDIUM":    { "models": [{ "provider": "anthropic", "model": "claude-sonnet-4-20250514" }
        "COMPLEX":   { "models": [{ "provider": "anthropic", "model": "claude-sonnet-4-20250514" }
        "REASONING": { "models": [{ "provider": "openai", "model": "o3" }] }
      },
      "boundaries": { "simple_medium": 0.25, "medium_complex": 0.50, "complex_reasoning": 0.75 },
      "context_messages": 3,
      "default_timeout_ms": 30000
    }
  }
}
```

Field	Type	Default	Description
enabled	bool	false	Master switch. When false, requests pass through to the MEDIUM tier default without classification.
tiers	HashMap<String, TierConfig>	{}	Per-tier model fallback chains. Keys: SIMPLE, MEDIUM, COMPLEX, REASONING.
weights	WeightConfig	(15 dimensions, summing to 1.0)	Classifier dimension weights.
boundaries	BoundaryConfig	0.25 / 0.50 / 0.75	Score boundaries between tiers. Must be ascending and in [0.0, 1.0].

Field	Type	Default	Description
`context_messages`	`usize`	3	Number of recent conversation messages included in classification.
`default_timeout_ms`	`u64`	30000	Default request timeout.

`TierConfig` has `models: Vec<TierEntry>` (ordered fallback list) and optional `timeout_ms`. `TierEntry` has `provider` and `model` string fields.

The `WeightConfig` has 15 dimension weights (token_count, vocabulary_complexity, syntax_complexity, domain_specificity, ambiguity, context_dependency, reasoning_depth, creativity_level, emotional_complexity, multimodality, instruction_complexity, knowledge_recency, code_complexity, mathematical_complexity, image_attachment). Weights are normalised to sum to 1.0 at load time.

For more on providers and models, see the Tutorial (`docs/howtos/tutorial.md`).

7.4 `agent`

Per-agent configuration overrides, keyed by agent name. Applied on top of built-in agent defaults.

```jsonc
{
  "agent": {
    "coder": {
      "model": "anthropic/claude-sonnet-4-20250514",
      "temperature": 0.2,
      "max_steps": 200,
      "skills": ["rust-review"],
      "permission": [
        { "permission": "file:write", "pattern": "src/**", "action": "allow" }
      ]
    }
  }
}
```

`AgentConfig` schema

Field	Type	Default	Description
`model`	`Option<String>`	None	Model in `"provider:model"` format.
`variant`	`Option<String>`	None	Agent variant selector.
`prompt`	`Option<String>`	None	System prompt override.
`temperature`	`Option<f32>`	None	Sampling temperature override.
`top_p`	`Option<f32>`	None	Top-p (nucleus) sampling override.
`mode`	`Option<String>`	None	Agent mode: `"primary"`, `"subagent"`, or `"all"`.
`hidden`	`bool`	false	Hide from user-facing listings.
`permission`	`Vec<PermissionRule>`	[]	Permission rules specific to this agent.

Field	Type	Default	Description
`max_steps`	`Option<u32>`	`None`	Maximum agentic loop iterations. Per-agent override of the loop step budget; when unset the `loop.max_steps` default (512) applies.
`skills`	`Vec<String>`	`[]`	Skill names to preload into the prompt.
`options`	`HashMap<String, Value>`	`{}`	Arbitrary agent-specific options.

For custom agents defined as JSON/Markdown files (OASF format), see `docs/howtos/custom-agents.md`.

7.5 `permission`

Global permission rules applied to all agents. Rules are evaluated last-match-wins using glob patterns.

```
{
  "permission": [
    { "permission": "file:write", "pattern": "src/**", "action": "allow" },
    { "permission": "file:write", "pattern": "secrets/**", "action": "deny" },
    { "permission": "bash", "pattern": "git push --force", "action": "deny" },
    { "permission": "*", "pattern": "/etc/**", "action": "deny" }
  ]
}
```

`PermissionRule` schema

Field	Type	Description
`permission`	`String` (parsed to `Permission`)	The permission type. Supports flat names (`read`, `bash`) and namespaced categories (`file:read`, `file:write`). These are permission types, not tool names: the rename left them unchanged.
`pattern`	`Option<String>`	Glob pattern matched against the resource path. `None` matches all.
`action`	`PermissionAction`	`allow`, `deny`, or `ask`.

Standard permission types

Name	Permission variant	Description
`read` / `file:read`	`Read`	Read access to files/resources.
`edit` / `write` / `file:write`	`Edit`	Write/edit access (the permission type, not the `Edit`/`Write` tool names).

Name	Permission variant	Description
`bash` / `execute` / `bash:execute`	`Bash`	Shell command execution (the permission type; the tool is `Bash`).
`web` / `fetch`	`Web`	Network/web access.
`question`	`Question`	Interactive question to user.
`plan_enter` / `plan`	`PlanEnter`	Enter planning phase (the tool is `EnterPlanMode`).
`plan_exit`	`PlanExit`	Exit planning phase (the tool is `ExitPlanMode`).
`task`	`Task`	Create/modify task items.
`external_directory`	`ExternalDirectory`	Access dirs outside project root.
`doom_loop`	`DoomLoop`	Detect/break infinite loops.
`*`	`Custom("*")`	Wildcard: applies to all permissions.
any other string	`Custom(String)`	User-defined permission type.

Evaluation order

1. **Permanent grants** (runtime "always allow" recorded via `PermissionChecker::record_always`) are checked first.
2. **Permission-specific rules** are evaluated last-match-wins.
3. **Wildcard rules** (`permission: "*"`) are evaluated last.
4. If no rule matches, the result is `ask`.

For the full permission system, YOLO mode, and autopilot, see `docs/howtos/permissions.md`.

7.6 `command`

User-defined slash-command shortcuts. Each entry maps a slash command name to a shell command.

```
{
  "command": {
    "build": { "command": "cargo build --release", "description": "Build release binary" },
    "test": { "command": "cargo test -- --nocapture", "description": "Run all tests with output"
  }
}
```

`CommandDef` schema

Field	Type	Description
`command`	`String`	Shell command to execute.
`description`	`String`	Human-readable description shown in help output.

7.7 `mcp`

MCP (Model Context Protocol) server definitions, keyed by server id.

```jsonc
{
  "mcp": {
    "filesystem": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-filesystem", "/home/user/docs"],
      "env": {}
    },
    "remote-api": {
      "type": "sse",
      "url": "https://mcp.example.com/sse",
      "headers": { "Authorization": "Bearer token123" },
      "disabled": false,
      "notification": "inject_summary"
    }
  }
}
```

`McpServerConfig` schema

Field	Type	Default	Description
`type`	`McpTransport`	`stdio`	Transport: `stdio`, `sse`, or `http`.
`command`	`Option<String>`	`None`	Executable path/name (stdio).
`args`	`Vec<String>`	`[]`	Command-line arguments (stdio).
`env`	`HashMap<String, String>`	`{}`	Environment variables (stdio).
`url`	`Option<String>`	`None`	URL endpoint (SSE/HTTP).
`headers`	`HashMap<String, String>`	`{}`	HTTP headers (SSE/HTTP).
`disabled`	`bool`	`false`	If `true`, server is configured but not started.
`notification`	`McpNotificationMode`	`none`	Push notification handling: `none`, `inject_summary`, `inject_and_run`.

7.8 `instructions`

Additional instruction strings appended to agent system prompts.

```jsonc
{
  "instructions": [
    "Always use rustfmt before writing Rust files.",
    "Prefer iterator adapters over manual loops."
  ]
}
```

Type: `Vec<String>`. Default: `[]`. These are appended to every agent's system prompt, merged (union) across config layers.

7.9 `skill_dirs`

Additional directories to scan for skill definitions, beyond the built-in locations.

```
{
  "skill_dirs": ["~/my-skills", "/shared/team-skills"]
}
```

Type: `Vec<String>`. Default: `[]`. Merged (union) across config layers.

7.10 `experimental`

Feature flags for experimental functionality.

```
{
  "experimental": {
    "open_telemetry": false,
    "parallel_tool_calls": false,
    "max_background_agents": 8,
    "background_agent_timeout": 3600
  }
}
```

`ExperimentalFlags` schema

Field	Type	Default	Description
`open_telemetry`	`bool`	`false`	Enable OpenTelemetry trace export (legacy; prefer `telemetry.otel`).
`parallel_tool_calls`	`bool`	`false`	Allow multiple tool calls from a single model turn to execute in parallel.
`max_background_agents`	`usize`	`8`	Maximum concurrent background sub-agent tasks.
`background_agent_timeout`	`u64`	`3600`	Timeout in seconds for background sub-agent tasks.

`open_telemetry` is mapped as a deprecated alias: if `telemetry.otel` is not explicitly enabled but `experimental.open_telemetry` is `true`, telemetry is enabled with default settings and a deprecation warning is emitted.

7.11 `hooks`

Lifecycle hooks. Currently a placeholder (`Vec<serde_json::Value>`). See `docs/howtos/hooks.md` for the hooks system documentation.

Type: `Vec<Value>`. Default: `[]`. Merged (append) across config layers.

7.12 `bash`

User-defined bash command allowlist and denylist additions.

```
{
  "bash": {
    "allowlist": ["curl", "wget"],
    "denylist": ["git push --force", "rm -rf /"]
  }
}
```

`BashConfig` schema

Field	Type	Default	Description
`allowlist`	`Vec<String>`	`[]`	Command prefixes exempted from the banned-command check (e.g. `"curl"`).
`denylist`	`Vec<String>`	`[]`	Substring patterns that always reject a command (e.g. `"git push --force"`).

Both lists are unioned (deduplicated) across global and project configs. The `/bash add` and `/bash remove` slash commands mutate these lists and persist changes to the targeted config file.

Allowlist entries match the first word of the command. **Denylist** entries are substring-matched anywhere in the command string.

This interacts with the 7-layer bash security model. See `docs/howtos/permissions.md` for the full model.

7.13 `dirs`

User-defined directory/file path allowlist and denylist additions.

```
{
  "dirs": {
    "allowlist": ["src/**", "tests/**", "*.rs"],
    "denylist": ["secrets/**", "/etc/**", ".env"]
  }
}
```

`DirsConfig` schema

Field	Type	Default	Description
`allowlist`	`Vec<String>`	`[]`	Glob patterns for paths automatically allowed (no prompt).
`denylist`	`Vec<String>`	`[]`	Glob patterns unconditionally rejected.
`allowed_roots`	`Vec<String>`	`[]`	*Additional* directory roots permitted for path-escape checking (multi-root / sub-agent access); the session's working directory is always allowed whether or not this list is set, and this list only extends it; canonicalized at load.

Both lists are unioned across config layers. **Denylist takes precedence over allowlist.** The `/dirs add` and `/dirs remove` slash commands mutate these lists.

Built-in denylist patterns (always active, not configurable) include system directories: `/bin/**`, `/sbin/**`, `/boot/**`, `/dev/**`, `/proc/**`, `/sys/**`, `/etc/**`, `/usr/bin/**`, `/usr/sbin/**`, `/usr/lib/**`, `/lib/**`, `/lib64/**`, and their macOS/Windows equivalents.

7.14 `code_index`

Persistent configuration for the codebase indexing subsystem.

```
{
  "code_index": {
    "enabled": true,
    "max_file_size": 1048576,
    "extra_exclude_dirs": ["vendor", "node_modules"],
    "extra_exclude_patterns": ["*.generated.rs"]
  }
}
```

`CodeIndexConfig` schema

Field	Type	Default	Description
`enabled`	`bool`	`true`	Whether code indexing is enabled.
`max_file_size`	`u64`	1048576 (1 MB)	Maximum file size in bytes to index.
`extra_exclude_dirs`	`Vec<String>`	`[]`	Additional directory names to exclude from scanning.
`extra_exclude_patterns`	`Vec<String>`	`[]`	Additional glob patterns to exclude.

The `enabled` field is serialised only when explicitly set (tracked by `specified.enabled`), so `/codeindex on`/`/codeindex off` toggles persist on save.

For the full code index guide, see `docs/howtos/codeindex.md`.

7.15 `stream`

LLM streaming behaviour: timeouts and retries.

```
{
  "stream": {
    "initial_response_timeout_secs": 300,
    "timeout_secs": 120,
    "max_retries": 4,
    "retry_backoff_secs": 2
  }
}
```

`StreamConfig` schema

Field	Type	Default	Validation	Description
`initial_response_timeout_secs`	u64	300	>= 5, >= `timeout_secs`	Seconds the HTTP client waits for the **first byte** of a streaming response.
`timeout_secs`	u64	120	>= 5	Seconds of silence between stream deltas before a stream is considered stalled.
`max_retries`	u32	4	<= 32	Maximum retry attempts after a stall or connection failure.
`retry_backoff_secs`	f64	2	—	Backoff multiplier per retry. Attempt N waits `N * retry_backoff_secs` seconds.

`validate()` returns a list of problems; the session processor refuses to start if any are present.

7.16 `memory`

Memory system configuration: blocks, structured store, retrieval, auto-extraction, decay, and cross-project sharing.

```
{
  "memory": {
    "enabled": true,
    "structured": { "enabled": true },
    "retrieval": {
      "max_memories_per_prompt": 100,
      "max_memory_tokens": 4000,
```

```
      "recency_weight": 0.3,
      "relevance_weight": 0.7
    },
    "auto_extract": {
      "enabled": false,
      "require_confirmation": true
    },
    "decay": {
      "factor": 0.95,
      "min_confidence": 0.1
    },
    "cross_project": {
      "enabled": false,
      "search_global": true,
      "project_override": true
    }
  }
}
```

`MemoryConfig` schema

Field	Type	Default	Description
`enabled`	`bool`	`true`	Master switch for the memory system.
`structured`	`StructuredMemoryConfig`	`{ "enabled": true }`	Structured store configuration.
`retrieval`	`RetrievalConfig`	(see below)	Retrieval configuration for prompt injection.
`auto_extract`	`AutoExtractConfig`	(see below)	Automatic memory extraction.
`decay`	`DecayConfig`	(see below)	Confidence decay.
`cross_project`	`CrossProjectConfig`	(see below)	Cross-project memory sharing.

Sub-schemas `StructuredMemoryConfig`: `enabled: bool` (default `true`).

`RetrievalConfig`:

Field	Type	Default	Description
`max_memories_per_prompt`	`usize`	`100`	Maximum structured memories fetched from SQLite for the system prompt. Acts as a safety cap.
`max_memory_tokens`	`Option<usize>`	`4000`	Approximate token budget for the injected memory section. `null` disables the budget (uses only the row cap).
`recency_weight`	`f64`	`0.3`	Weight for recency when ranking (0.0-1.0).
`relevance_weight`	`f64`	`0.7`	Weight for relevance when ranking (0.0-1.0).

`AutoExtractConfig`:

Field	Type	Default	Description
`enabled`	`bool`	`false`	Whether automatic memory extraction is enabled.
`require_confirmation`	`bool`	`true`	Whether extracted candidates require explicit confirmation before storage.

`DecayConfig`:

Field	Type	Default	Description
`factor`	`f64`	`0.95`	Multiplicative decay factor per day since last access. 1.0 disables decay.
`min_confidence`	`f64`	`0.1`	Minimum confidence floor. Memories never decay below this.

`CrossProjectConfig`:

Field	Type	Default	Description
`enabled`	`bool`	`false`	Whether cross-project memory sharing is enabled.
`search_global`	`bool`	`true`	Whether search includes global memories.
`project_override`	`bool`	`true`	Whether project-specific blocks override global blocks with the same label.

7.17 `compaction`

Context-window compaction configuration (OpenCode-derived summarisation).

```
{
  "compaction": {
    "auto": true,
    "threshold": 0.7,
    "buffer": 0.10,
    "keep": { "tokens": 0.20 },
    "model": { "provider_id": "ollama", "model_id": "qwen2.5:1.5b" },
    "summary_tokens": 1500,
    "tool_output_max_chars": 2000
  }
}
```

`CompactionConfig` **schema**

Field	Type	Default	Description
`auto`	`bool`	`true`	Whether automatic pre-send compaction is enabled. When `false`, only provider context-overflow errors trigger emergency compaction.
`threshold`	`Option<f64>`	`Some(0.7)`	Fraction of context window at which to trigger compaction (0.0-1.0). `None` falls back to buffer-based trigger. Clamped to minimum 0.7 so routine prompts under 70% never trigger compaction.
`buffer`	`f64`	`0.10`	Token buffer as a fraction of context window. When `threshold` is `None`, triggers when tokens exceed `context_window - max(output_tokens, context_window * buffer)`.
`keep`	`KeepConfig`	`{ tokens: Some(0.20) }`	Recent turns kept verbatim after compaction.
`model`	`Option<CompactionModelRef>`	`None`	Optional fast/cheap model override (`provider_id` + `model_id`) for the `/compact` summarisation call. The session's primary model is used when unset, and the pre-send auto-compaction path always uses the session's primary model (an `AgentNotice` is published when an override is configured).
`summary_tokens`	`usize`	`1500`	Maximum tokens requested for the compaction summary output.
`tool_output_max_chars`	`usize`	`2000`	Truncation limit for each tool output serialised into the summarisation prompt.

`KeepConfig`: `tokens: Option<f64>` (default `Some(0.20)`) — max fraction of context window preserved as the verbatim tail.

Runtime behaviour: - The summarisation prompt is capped adaptively at `min(60,000 chars, context_window * 4 / 2)` so small local models get a proportionally smaller prompt. - The compaction stream has a hard 180-second overall cap plus a 60-second per-chunk stall timeout; progress is reported every 10 seconds with the elapsed time and the summary character count received so far. - The compaction agent's configured temperature is honoured by the summarisation request. - Cancellation is checked between stream chunks, so `/compact` honours the session cancel flag during the LLM call. - Token estimation uses an allocation-free byte counter instead of re-serialising tool JSON schemas.

7.18 `gitlab`

GitLab integration configuration. Values set here override those stored in the ragent database (set via `/gitlab setup`). Environment variables take the highest priority.

```
{
  "gitlab": {
    "instance_url": "https://gitlab.example.com",
    "token": "glpat-xxxxxxxxxxx",
    "username": "myuser"
  }
}
```

`GitLabIntegrationConfig` schema

Field	Type	Default	Description
`instance_url`	`Option<String>`	None	GitLab instance base URL.
`token`	`Option<String>`	None	Personal Access Token.
`username`	`Option<String>`	None	GitLab username / identity.

Env var overrides: `GITLAB_TOKEN`, `GITLAB_URL`, `GITLAB_USERNAME`.

7.19 `yolo`

YOLO mode bypasses all command validation and tool restrictions.

```
{ "yolo": false }
```

Type: `bool`. Default: `false`.

When enabled, the following safety checks are skipped: - Bash denied patterns (destructive commands like `rm -rf /` are allowed) - Dynamic context allowlist (any executable can run in skill bodies) - MCP config validation (shell metacharacters and unvalidated paths permitted)

Warning: This is inherently dangerous. Use only when you trust the agent and its inputs completely, or for local development/debugging.

The `Alt+Y` keybinding and `/yolo` slash command toggle this at runtime and persist the change to the **user-global** config (`~/.config/ragent/ragent.json`); a project-local `yolo` value is ignored (untrusted repository content, SECTASKS T-011). See `docs/howtos/permissions.md`.

The persistence path writes only the single top-level `yolo` key in the raw global file (`Config::set_global_bool_key`), preserving every other key byte-for-byte rather than re-serialising the merged `Config`, and the write is atomic (unique temp file + `fsync` + rename), so an interrupted write can never leave a partial config that a later `Config::load` would fall back on (which would silently reset a persisted `true`). Every such write logs the key, value, and path with a forced backtrace, so a privileged flag flip is attributable.

7.20 `edit_log`

Edit-operation logging for `Edit` and `multi_edit` tools.

```
{ "edit_log": false }
```

Type: `bool`. Default: `false`.

When enabled, all edit operations are logged. The `Alt+E` keybinding and `/editlog` slash command toggle this at runtime and persist the change.

7.21 `prices`

User-defined price overrides for cost estimation. Each entry overrides the built-in price table for a specific model. Prices are in USD per 1,000,000 tokens.

```
{
  "prices": [
    { "model": "gpt-4o", "input_per_1m": 2.50, "output_per_1m": 10.00 },
    { "model": "claude-sonnet-4-20250514", "input_per_1m": 3.00, "output_per_1m": 15.00 }
  ]
}
```

`PriceEntry` schema

Field	Type	Description
`model`	`String`	Model identifier as returned by the provider.
`input_per_1m`	`f64`	Price per 1M input/prompt tokens (USD).
`output_per_1m`	`f64`	Price per 1M output/completion tokens (USD).

When merged, overlay entries replace base entries with the same model id (last-wins per model).

7.22 `telemetry`

OpenTelemetry metrics export configuration.

```
{
  "telemetry": {
    "otel": {
      "enabled": false,
      "endpoint": "http://localhost:4318",
      "protocol": "http",
      "export_interval_seconds": 30,
      "export_timeout_seconds": 10,
      "service_name": "ragent",
      "resource_attributes": { "deployment.environment": "production" },
      "metrics": { "ragent.tool.call_count": false },
      "internal_port": null,
```

```
      "cardinality_limit": 1000,
      "logs": { "enabled": false, "level": "info" }
    }
  }
}
```

`TelemetryConfig` schema Wraps `OtelConfig` as `telemetry.otel`.

`OtelConfig` schema

Field	Type	Default	Description
`enabled`	`bool`	`false`	Master on/off switch.
`endpoint`	`String`	`"http://localhost:4318"`	OTLP endpoint base URL. Must be `http://` or `https://`. ragent appends the OTLP/HTTP signal path (`/v1/metrics`) to a base URL, so `http://localhost:4318` exports to `http://localhost:4318/v1/metrics`; a base path (`https://gw.example.com/otel`) is kept. An endpoint that already names a signal path (`.../v1/metrics`) is used exactly as written.
`protocol`	`OtelProtocol`	`http`	Transport: `http` or `grpc`.
`export_interval_seconds`	`u64`	`30`	Batch export interval. Must be > 0 when enabled.
`export_timeout_seconds`	`u64`	`10`	Per-export request timeout. Must be > 0 when enabled.
`service_name`	`String`	`"ragent"`	`service.name` resource attribute. Must not be empty.
`resource_attributes`	`HashMap<String, String>`	`{}`	Custom resource attributes appended to every metric.
`metrics`	`HashMap<String, bool>`	`{}`	Per-metric enable/disable toggles. Absent = enabled by default.
`internal_port`	`Option<u16>`	`None`	Optional in-process Prometheus text endpoint port.
`cardinality_limit`	`usize`	`1000`	Max distinct attribute combinations per metric before overflow into `unknown` bucket.
`logs`	`OtelLogsConfig`	`{ enabled: false, level: "info" }`	OTLP log-signal export of ragent's `tracing` events (see below).

`OtelLogsConfig` schema (`telemetry.otel.logs`) Ragent logs through `tracing` to a local sink: the TUI log panel in a terminal session, or stderr in a headless/server run. When telemetry is on, a **second** destination - the same OTLP collector that receives the metrics - is fed from this block, without redirecting

the local sink. There is no separate logs switch: `/telemetry on` writes `enabled: true` here alongside `telemetry.otel.enabled`, `/telemetry off` clears it, and `/telemetry logs [<level>]` only changes how much is exported.

Field	Type	Default	Description
`enabled`	`bool`	`false`	Ship `tracing` events over the OTLP logs signal. Maintained by `/telemetry on|off` (which sets it together with `telemetry.otel.enabled`); a hand-edited `true` still requires `telemetry.otel.enabled: true` to export anything. The setup dialog prefills `Logs level` with the current level (or `info`); a blank field keeps the default rather than clearing this flag - `/telemetry on|off` is the only switch.
`level`	`String`	`"info"`	Minimum severity exported: `trace`, `debug`, `info`, `warn` or `error`. A value outside that list is a validation problem; an unknown value falls back to `info`. Events below the level still reach the local sink.

The logs signal travels on the same `endpoint`, `protocol` and `export_timeout_seconds` as the metrics, posting to the `/v1/logs` sibling of the metrics path (so `http://localhost:4318` sends logs to `http://localhost:4318/v1/logs`), batches on the same `export_interval_seconds`, and obeys the same TLS rule for gRPC (a cleartext non-loopback endpoint is refused). Every exported body is passed through the shared credential redactor and every exported attribute through the metric-attribute sensitive-data guard; a body longer than 16 KiB is truncated with a `... [truncated]` marker. If the log exporter cannot be built, the failure is logged at `warn` level and the metric signal keeps running.

```
{
  "telemetry": {
    "otel": {
      "enabled": true,
      "endpoint": "http://localhost:4318",
      "logs": { "enabled": true, "level": "warn" }
    }
  }
}
```

The legacy `experimental.open_telemetry` flag is mapped as a deprecated alias via `apply_legacy_flag()`.

7.23 `agent_perf`

Agent-loop performance configuration.

```jsonc
{
  "agent_perf": {
    "enabled": true,
    "profiling": false,
    "step_budget_secs": 300,
    "stall_timeout_secs": 60,
    "max_concurrent_tools": 4,
    "parallel_independent_tools": true
  }
}
```

`AgentPerfConfig` schema

Field	Type	Default	Validation	Description
`enabled`	`bool`	`true`	—	Master switch. When `false`, all performance optimisations short-circuit.
`profiling`	`bool`	`false`	—	Emit detailed per-scope timing logs at `info` level.
`step_budget_secs`	`u64`	300	>= 5	Maximum wall-clock seconds per agent step.
`stall_timeout_secs`	`u64`	60	>= 5	Maximum seconds without a stream delta before stall recovery fires.
`max_concurrent_tools`	`u32`	`min(available_parallelism, 4)`	>= 1	Maximum parallel tool calls per turn.
`parallel_independent_tools`	`bool`	`true`	—	Execute independent tool calls in parallel.

`validate()` returns a list of problems; the agent loop refuses to start if any are present.

7.24 Search API keys

Several API keys for the `mf_search` / `websearch` tools can be stored in the config file. Environment variables always take precedence.

Field	Type	Default	Env var override	Description
`tavily_api_key`	`Option<String>`	None	`TAVILY_API_KEY`	Tavily search API key for the `websearch` tool.
`langsearch_api_key`	`Option<String>`	None	—	LangSearch API key for `mf_search`. Masked in diagnostics.

Field	Type	Default	Env var override	Description
`perplexity_api_key`	`Option<String>`	None	—	Perplexity Sonar API key for `mf_search`. Masked in diagnostics.
`exa_api_key`	`Option<String>`	None	`EXA_API_KEY`	Exa Search API key for `mf_search`. Masked in diagnostics.
`serper_api_key`	`Option<String>`	None	`SERPER_API_KEY`	Serper (Google Search) API key for `mf_search`. Masked in diagnostics.
`openalex_email`	`Option<String>`	None	`OPENALEX_EMAIL`	OpenAlex polite-pool email for `mf_search`. Masked in diagnostics.

When present, `mf_search` queries the corresponding backend as an additional search engine alongside the keyless backends (OpenAlex, Wikipedia).

7.25 `sdd`

Spec-Driven Development (SDD) capability toggles. All flags default to `false` (opt-in).

```
{
  "sdd": {
    "clarification_markers": true,
    "quality_checklists": true,
    "constitution": true,
    "phase_minus_one_gates": true,
    "branch_per_spec": true,
    "research_artifacts": true,
    "data_model": true,
    "contracts": true,
    "quickstart": true,
    "test_first_ordering": true,
    "consistency_checks": true
  }
}
```

`SddConfig` schema

Field	Type	Default	Description
`clarification_markers`	`bool`	`false`	Enable `[NEEDS CLARIFICATION]` marker detection (FR-002).
`quality_checklists`	`bool`	`false`	Embed quality checklists in spec/plan templates (FR-006).

Field	Type	Default	Description
`constitution`	`bool`	`false`	Generate and parse `CONSTITUTION.md` (FR-007).
`phase_minus_one_gates`	`bool`	`false`	Enable Phase -1 pre-implementation gate validation (FR-008).
`branch_per_spec`	`bool`	`false`	Create a git branch per spec (FR-009).
`research_artifacts`	`bool`	`false`	Link research artifacts into SPEC.md frontmatter (FR-010).
`data_model`	`bool`	`false`	Generate `data-model.md` during `/spec plan` (FR-011).
`contracts`	`bool`	`false`	Generate `contracts/` directory during `/spec plan` (FR-012).
`quickstart`	`bool`	`false`	Generate `quickstart.md` validation scenarios (FR-013).
`test_first_ordering`	`bool`	`false`	Enforce test-first file creation ordering in plans (FR-014).
`consistency_checks`	`bool`	`false`	Run ambiguity, contradiction, and gap consistency checks (FR-015).

Merge uses OR semantics: a flag enabled in either base or overlay stays enabled.

For the spec system, see `docs/howtos/spec.md`.

7.26 `trigger`

Dynamic trigger rule system configuration.

```
{
  "trigger": {
    "enabled": true,
    "poll_interval_secs": 30,
    "max_rules": 32
  }
}
```

`TriggerConfig` schema

Field	Type	Default	Description
`enabled`	`bool`	`true`	Master feature gate. When `false`, all trigger functionality no-ops.
`poll_interval_secs`	`u64`	`30`	Interval at which dynamic trigger rules poll their conditions.
`max_rules`	`usize`	`32`	Maximum number of dynamic trigger rules per session.

`is_empty()` returns `true` when all fields are at default values, so the section is omitted from serialised output when nothing is configured.

7.27 `piegap`

Pie feature gap toggles. Each flag gates a standalone pie-derived feature. All default to `false` (opt-in).

```
{
  "piegap": {
    "triggers": true,
    "mcp_notifications": true,
    "inbox": true,
    "hooks": true,
    "archive": true,
    "bug_report": true,
    "templates": true,
    "goal": true,
    "web_ui": true,
    "undo": true,
    "session_naming": true
  }
}
```

`PieGapConfig` schema

Field	Type	Default	Description
`triggers`	`bool`	`false`	Enable dynamic trigger rules (G-01).
`mcp_notifications`	`bool`	`false`	Enable MCP notification push events (G-02).
`inbox`	`bool`	`false`	Enable stateful loops + triage inbox (G-03).
`hooks`	`bool`	`false`	Enable lifecycle hooks (G-04).
`archive`	`bool`	`false`	Enable portable session archive export/import (G-05).
`bug_report`	`bool`	`false`	Enable bug report generation (G-06).
`templates`	`bool`	`false`	Enable reusable prompt templates (G-07).
`goal`	`bool`	`false`	Enable goal-based autonomous stop hook (G-10).
`web_ui`	`bool`	`false`	Enable browser-based web UI (G-12).
`undo`	`bool`	`false`	Enable `/undo` slash command (G-13).
`session_naming`	`bool`	`false`	Enable session naming (G-14).

Merge uses OR semantics. `is_empty()` returns `true` when no flags are enabled.

7.28 `research`

Research subsystem configuration.

```
{
  "research": {
    "open_access_recovery": true,
    "exclude_academic_engines": false,
    "contact_email": "user@example.com",
    "oa_min_full_text_chars": 1000,
    "max_concepts": 5,
    "max_findings": 20,
    "evaluate": {
      "enabled": true
    }
  }
}
```

`ResearchConfig` schema

Field	Type	Default	Description
`open_access_recovery`	`bool`	`false`	Enable open-access recovery via Unpaywall and Europe PMC for short scholarly sources (FR-011).
`exclude_academic_engines`	`bool`	`false`	Keep academically-classified search engines (OpenAlex) excluded from research runs (spec `researchnoacc` FR-012). Scholarly engines are already excluded by default; a per-run `--papers` opts them back in and takes precedence.
`contact_email`	`Option<String>`	`None`	Contact email required by Unpaywall's terms of service (FR-012).
`oa_min_full_text_chars`	`usize`	`1000`	Minimum full-text length (chars) that triggers OA recovery.
`max_concepts`	`usize`	`5`	Maximum number of concepts rendered in the report's `## Concepts` block (spec `researchmax` FR-008). A per-run `--max-concepts` takes precedence; `0` means unbounded.
`max_findings`	`usize`	`20`	Maximum number of findings rendered in the report's `## Findings` block (spec `researchmax` FR-008). A per-run `--max-findings` takes precedence; `0` means unbounded.

Field	Type	Default	Description
`evaluate`	`ResearchEvaluateConfig`	`{"enabled": false}`	Self-evaluation scorecard settings (FR-015 of specs/opendeepresearch). When `enabled`, the research pipeline appends a deterministic quality scorecard (quality, relevance, groundedness, completeness, structure) to the report.

As of v1.0.77, `evaluate` can be configured in `ragent.json` and is omitted from serialized output when at its default value (`enabled: false`).

As of v1.0.100, the default web-phase timeout is 180 seconds and can be overridden per run with `--web-time N` (or `--web-phase-timeout-secs N`); `0` disables the deadline. When the deadline elapses, the run continues with the sources gathered so far. See `docs/howtos/research.md` for the full research workflow.

`open_access_recovery` uses OR semantics on merge. `contact_email` and `oa_min_full_text_chars` override base when present. `is_empty()` returns `true` when at default values.

`exclude_academic_engines` also uses OR semantics on merge and is omitted from serialized output when `false`. Scholarly engines are excluded from research runs by default; when enabled, the setting makes that exclusion explicit and research runs pass `exclude_engines = ["openalex"]` to `mf_search` so OpenAlex is never queried; a per-run `--papers` takes precedence over the config value. See `docs/howtos/research.md`.

A per-run `--oa-enable` / `--no-oa` flag overrides `research.open_access_recovery`: the flag wins when set, otherwise the config value is used, and when neither is set recovery is off. See `docs/howtos/research.md`.

For the research system, see `docs/howtos/research.md`.

7.29 `loop`

Goal-driven `/loop` run budgets and checkpoints (see `docs/howtos/loopprogramming.md`). The whole section is omitted from serialized output when every field is at its default.

```
{
  "loop": {
    "max_steps": 512,
    "cost_limit": null,
    "error_retry_allowance": 3,
    "checkpoints": true,
    "checkpoint_timeout_secs": 120
  }
}
```

`LoopConfig` schema

Field	Type	Default	Description
`max_steps`	`u32`	`512`	Default step budget for a loop run (`DEFAULT_LOOP_MAX_STEPS`). A per-agent `agent.<name>.max_steps` override takes precedence.
`cost_limit`	`Option<u64>`	`None`	Token-cost budget (accumulated input + output) across a loop run; `None` means no gate.
`error_retry_allowance`	`u32`	`3`	Consecutive recoverable-failure tolerance before the loop stops with status `error`.
`checkpoints`	`bool`	`true`	Force destructive-action checkpoints before destructive tool calls even when an allow rule would auto-approve. Merge is monotonic: once disabled it stays disabled.
`checkpoint_timeout_secs`	`u32`	`120`	Seconds a forced checkpoint prompt waits for a reply before the timeout counts as denial.

Merge is sentinel-based: overlay fields equal to the compiled default keep the base (global) values; only fields the overlay configured away from the default win.

7.30 `activity_log`

```
{
  "activity_log": true
}
```

Field	Type	Default	Description
`activity_log`	`bool`	`true`	Record agent events to the activity-log SQLite store (append-only JSONL-style event log with `RunId`/`EventId` identifiers).

Toggle at runtime with `/alog on|off`.

7.31 `plugins`

Plugin subsystem configuration (spec `plugins`). Controls the master switch, the sandbox budgets, the store location, and per-plugin permission grants.

```
{
  "plugins": {
    "enabled": true,                // master switch; default true
```

```
    "max_execution_ms": 5000,        // per-tool wall-clock budget
    "max_entry_ms": 10000,           // entry-point wall-clock budget
    "max_memory_mb": 64,             // per-JS-context memory ceiling
    "store_dir": null,               // optional override of the plugin store path
    "permissions": {                 // optional per-plugin permission grants
      "codex-weather": ["network.outbound"]
    }
  }
}
```

Field	Type	Default	Description
`enabled`	`bool`	`true`	Master switch. When `false`, no discovery or loading happens and `/plugins` subcommands other than `help` report the subsystem is disabled.
`max_execution_ms`	`u64`	`5000`	Per-plugin-tool wall-clock execution budget in milliseconds.
`max_entry_ms`	`u64`	`10000`	Wall-clock budget for executing a plugin's entry point in milliseconds.
`max_memory_mb`	`u64`	`64`	Per-JavaScript-context memory ceiling in mebibytes.
`store_dir`	`string?`	`null`	Optional override of the plugin store directory. When `null`, the store is discovered at `.ragent/plugins/` (project), falling back to `~/.config/ragent/plugins/` (user-global).
`permissions`	`map<string, string[]>`	`{}`	Per-plugin permission grants keyed by plugin id.

The section merges with overlay-wins precedence (project config overrides user-global). Manage plugins with `/plugins list|add|remove|enable|disable|test|help` in the TUI or `ragent plugins <sub>` from the CLI. See `docs/howtos/slashcommands/plugins.md`.

7.32 `input_queue_capacity`

Maximum number of messages the TUI message input queue may hold (spec `inputqueue`). While the primary agent is executing, messages submitted with `Enter` are staged in this bounded FIFO queue and run at each turn boundary; a two-digit counter appears before the prompt while entries are pending.

```
{
  "input_queue_capacity": 32
}
```

Field	Type	Default	Description
`input_queue_capacity`	`usize?`	`32`	Maximum queued entries; resolved by `Config::effective_input_queue_capacity()` and clamped to `1..=99` (the two-digit counter's bound). Submissions beyond the cap are rejected with a status message and the typed text/attachments are restored.

The field merges overlay-wins: an explicitly set project value survives an absent user-global value. The per-run queue is in memory only and is never persisted. Manage the queue with the `Alt+Q` menu or the `/queue` slash command (see `docs/howtos/slashcommands/queue.md` and `TUI-QUICKSTART.md` §4).

7.33 `connectors`

Connector subsystem configuration (spec `connectors`). Controls the master switch, the connector store location, the catalogue endpoints and fetch budgets, and the non-secret credential-name mapping.

```
{
  "connectors": {
    "enabled": true,                        // master switch; default true
    "store_dir": null,                      // optional override of the connector store path
    "stores": {                             // catalogue endpoints
      "community": { "url": "https://example.org/connectors/index.json" },
      "timeout_ms": 10000,
      "max_index_bytes": 2097152,
      "cache_ttl_secs": 3600
    },
    "credentials": {                        // non-secret credential-name mapping
      "google-drive": { "token": "GDRIVE_TOKEN" }
    }
  }
}
```

Field	Type	Default	Description
`enabled`	`bool`	`true`	Master switch. When `false`, the connector subsystem performs no discovery, no catalogue fetch, and no connection, and `/connectors` subcommands other than `help` report that the system is disabled.
`store_dir`	`string?`	`null`	Optional override of the connector store directory. When `null`, the store is discovered at `.ragent/connectors/` (project), falling back to `~/.config/ragent/connectors/` (user-global).

Field	Type	Default	Description
`stores`	`object?`	`null`	Catalogue endpoints and shared fetch budgets. When `null`, the compiled default endpoints and budgets apply.
`stores.<name>`	`object`	`{}`	One named catalogue endpoint, for example `claude` or `community`.
`stores.<name>.url`	`string?`	`null`	Catalogue-index `https://` URL. A `null`, empty, or whitespace-only value falls back to the compiled default endpoint for that catalogue; a non-`https` or host-less value is refused when the catalogue is fetched.
`stores.timeout_ms`	`u64`	`10000`	Per-fetch wall-clock budget in milliseconds, shared by every catalogue.
`stores.max_index_bytes`	`u64`	`2097152`	Maximum accepted catalogue-index size in bytes; a larger index aborts the fetch, shared by every catalogue.
`stores.cache_ttl_secs`	`u64`	`3600`	Time-to-live for a cached catalogue in seconds; `0` disables the index cache.
`stores.marketplaces`	`array`	`[]`	Additional catalogues the `/connectors claude` browser fetches and merges. Each entry has a `store` (the catalogue token), a display `name`, an optional `url`, and the `source` it was added from. Written to the **user-global** config by `/connectors marketplace claude add`; a non-`https` `url` is refused by the scheme guard.
`credentials`	`map<string, object>`	`{}`	Non-secret credential-name mapping keyed by connector id. Each value holds the **name** of a credential in the encrypted credential store, never the secret value.

The section merges with overlay-wins precedence (project config overrides user-global). Manage connectors with `/connectors list|claude|add|remove|enable|disable|connect|disconnect|auth|test|stores|mark` in the TUI or `ragent connectors <sub>` from the CLI.

7.34 `execution_backend` and `backends`

Select **where** a session's tool invocations run (spec `openhands` FR-001, FR-026). When unset, ragent executes tools on the host exactly as before (`local`, FR-019). A `docker` or `podman` selection runs the shell, file, and search tools inside a container instead. A `remote` selection relays the whole turn to a second ragent server over its REST+SSE API and mirrors that server's event stream into the local TUI

(FR-021).

```
{
  // Terse form: select a kind by label.
  "execution_backend": "podman",
  // Registry form: declare backends and select one by id (or by kind label).
  "backends": [
    { "id": "box", "name": "Podman sandbox", "kind": "podman",
      "image": "ghcr.io/yourorg/ragent-sandbox:latest", "workspace": "/projects" },
    { "id": "build-farm", "name": "Remote build host", "kind": "remote",
      "url": "http://10.0.0.5:9100", "api_key": "..." }
  ]
}
```

The `execution_backend` key accepts either a bare label (`local`, `docker`, `podman`, `remote`) or an inline descriptor object (`{ "id", "kind", "image", "workspace" }`). An object that names a registered `id` (with no `kind`) resolves its kind from the matching `backends` entry. An absent or unknown value resolves safely to `local`.

`BackendConfig` schema

Field	Type	Default	Description
`id`	`String`	`""`	Stable identifier used to select this backend.
`name`	`Option<String>`	`None`	Human-readable display name; defaults to `id`.
`kind`	`String`	`""`	Backend kind: `local`, `docker`, `podman`, or `remote`.
`image`	`Option<String>`	`None`	Container image reference (`docker`/`podman`).
`workspace`	`Option<String>`	`None`	Container-side workspace mount point. An absolute path overrides the default `/projects`; a relative value such as `"."` keeps the default.
`url`	`Option<String>`	`None`	Base URL of a remote server (`remote` kind; T-003).
`api_key`	`Option<String>`	`None`	API key for a remote server (`remote` kind; T-003). Redacted in diagnostics.
`credentials`	`Vec<String>`	`[]`	Names of encrypted-store credentials to inject at spawn time (T-006, FR-010). A name is not a secret: the value is resolved from the encrypted credential store when the sandbox is provisioned and injected as a container environment variable.

Container isolation. The workspace is a container-named volume, never a host bind mount, so a sandboxed tool cannot read or write the host working directory (FR-020, FR-035). A provisioning failure

(a missing runtime, or a missing or unpullable `image`) fails the turn with a provisioning error and never runs the tool on the host (FR-031). `podman` is the default container runtime when no runtime is specified (FR-026), and a container backend is only offered when a runtime is present on `PATH`.

Secret injection (FR-010). A `docker`/`podman`/`remote` descriptor that lists `credentials` has each named value resolved from the encrypted credential store (the same store `/auth` writes to) **at spawn time** and injected as a container environment variable (`-e NAME=VALUE`); a remote descriptor's first named credential is used as its bearer key. A name the store does not hold is a provisioning failure - the sandbox is never started without a required credential. The value is never written into `ragent.json`, the container image, or the workspace (FR-035). A literal `api_key` on a remote descriptor still works and takes precedence, so existing configs are unchanged.

Workspace persistence (FR-023). The workspace is a named volume derived from the project identity, so it survives replacement of the container. Changing the `credentials` list re-provisions the container (a fresh environment) while re-mounting the same volume, so the conversation's files are preserved.

Remote backend. A `remote` descriptor needs a `url` (the base URL of a second ragent server, for example `ragent serve --port 9100`) and an `api_key` (the server's bearer token). While the remote backend is active, the session processor does **not** run tools locally: it opens (or reuses) a session on the remote server, posts the prompt to `POST /sessions/{id}/messages`, and mirrors the server's `text/event-stream` events into the local event bus so the turn renders exactly like a local one (FR-021). If the remote server cannot be reached before the turn, or drops the stream part-way through it, the turn fails with a connection error shown in the TUI and none of that turn's tools are re-executed locally (FR-031, FR-034). A remote backend with no `url` fails at provisioning rather than falling back to host execution.

Backend registry (FR-004, FR-030). Every configured backend is exposed as a durable registry entry carrying a stable `id`, a display name, a kind, a secret-free connection descriptor, and a health state. The config file is the durable record; `crates/ragent-agent/src/backend/registry.rs` rebuilds the read model from it. The built-in `local` entry is always present and healthy (FR-019); a `docker`/`podman` entry is `ok` when its runtime is on `PATH`, else `unavailable`; a `remote` entry is `ok` when a `url` and a key are configured (FR-030), and a live `/health` probe refines that state (the probe targets the server's **public** `/health` endpoint, so no key is sent). The TUI backend surface (T-005, FR-008) renders this registry: the active backend, each registered backend's health, and a switcher.

7.35 `acp`

Agent Client Protocol configuration (spec `openhands` FR-015, FR-022, FR-028, FR-036). Two roles:

ACP client (`agents`, `default_agent`). When `agents` is present, ragent can drive an external coding agent (Claude Code, Codex, Gemini CLI, or a custom ACP server) as a subprocess over JSON-RPC on stdio. Absent by default, which leaves the client subsystem inert.

ACP server (`server_enabled`, `server_agent`). When `server_enabled` is `true`, ragent **serves** ACP-capable editors (Zed, VS Code, JetBrains) over JSON-RPC on its own stdio. Disabled by default. Serving additionally requires a binary built with the `acp-server` Cargo feature, and the endpoint is reachable only through the `ragent acp-server` subcommand (not via the TUI or `run`):

```jsonc
{
  "acp": {
    "server_enabled": true,
    "server_agent": "coder",
    "agents": {
      "claude": { "id": "claude", "command": "claude-code-acp", "args": [] }
```

```
    }
  }
}
```

- `server_enabled` (bool, default `false`) - master switch for the server endpoint. When `false` (or the feature is not compiled in), `ragent acp-server` exits with a usage error (exit code 2) and serves nothing.
- `server_agent` (string, optional) - the local ragent agent the server drives. Resolves `server_agent` -> `default_agent` -> the top-level `defaultAgent`.

Build and run:

```
cargo build --features acp-server
ragent acp-server            # editor attaches over stdin/stdout
```

Point an editor at the binary with `ragent acp-server` as the agent command.

See `docs/howtos/acp.md` for the full guide: the client turn lifecycle, how an agent binds to an ACP entry, worked examples for Claude Code, Codex, and Gemini CLI, the server endpoint, and troubleshooting.

7.36 `i18n`

UI internationalisation (spec `openhands` FR-027). Opt-in and absent by default; when present, user-facing strings render from the configured locale's message catalog and fall back to English for any missing key.

```
{ "i18n": { "enabled": true, "locale": "fr" } }
```

7.37 `security_analyzer`

LLM-based security-analyzer permission mode (spec `openhands` FR-006, FR-016, FR-017). Opt-in and absent by default; when present and enabled, each proposed tool action is evaluated by an LLM that returns an allow/ask/deny verdict with a rationale before the action is permitted or refused. See `docs/howtos/llmsecurity.md` for the full guide.

7.38 `openai`

OpenAI-compatible inbound HTTP surface configuration (spec `openhands` FR-012, FR-024, FR-032). Opt-in and absent by default; the `/v1/models` and `/v1/chat/completions` endpoints are always mounted behind the native bearer-token check, and this section carries the surfaced token and arms the refuse-to-start gate.

```
{
  "openai": {
    "enabled": true,
    "token": "sk-ragent-local"
  }
}
```

Field	Type	Default	Description
`enabled`	`bool`	`false`	Whether the OpenAI-compatible surface is an explicitly enabled capability. Enabling it arms the FR-032 token gate.
`token`	`Option<String>`	`None`	Bearer token required for every request to the surface (FR-024). Redacted in diagnostics. Falls back to the ambient `RAGENT_TOKEN`.

Refuse to start without a token (FR-032). The surface is never served unauthenticated: when `enabled: true` but no `token` (or `RAGENT_TOKEN`) is available, `ragent serve` exits with an error and starts no server. A configured token is the server's single bearer token (FR-024) - ragent does not run a second credential for `/v1`. Only the top-level (trusted) config may set the token; a project-local `.ragent/ragent.json` overlay has its `openai` section dropped by `merge_project`, so repository content cannot choose the credential that guards the whole API.

Streaming (FR-012). `POST /v1/chat/completions` with `stream:true` streams `text/event-stream` chunks in the OpenAI `chat.completion.chunk` shape and terminates with `data: [DONE]`; `stream:false` returns one `chat.completion` JSON object (FR-011). Both paths preserve the permission checks (FR-038).

7.39 `automation`

Automation service configuration (spec `openhands` FR-013, FR-014, FR-018, FR-033). Absent by default; the service is inert unless a present block sets `enabled: true`.

```
{
  "automation": {
    "enabled": true,
    "run_history_cap": 500,
    "scheduler_tick_secs": 30,
    "output_dir": null,
    "automations": [
      {
        "id": "on-issue",
        "name": "Issue triage",
        "agent": "general",
        "prompt": "Triage issue {{payload}}",
        "trigger": { "kind": "webhook" },
        "backend": "local",
        "dispatch": [
          { "kind": "slack", "url": "https://hooks.slack.com/...", "token_env": "SLACK_BOT_TOKEN"
        ],
        "enabled": true
      },
      {
```

```
        "id": "nightly",
        "trigger": { "kind": "schedule", "schedule": "every 2m" },
        "backend": "local"
      }
    ]
  }
}
```

Field	Type	Default	Description
`enabled`	`bool`	`true` (only when the block is present)	Master switch. An **absent** block leaves the service inert; `Config::automation_enabled` reads presence.
`run_history_cap`	`usize`	`500`	Run-history records retained per automation; older records are pruned.
`scheduler_tick_secs`	`u64`	`30`	Scheduler tick interval in seconds.
`output_dir`	`Option<PathBuf>`	`None`	Override for the run-output directory (default `<working_dir>/log/automation/`).
`automations`	`Vec<AutomationDefinition>`	`[]`	The registered automations.

Each automation definition:

Field	Type	Default	Description
`id`	`String`	required	Unique id; also the webhook URL segment (`POST /auto/<id>`).
`name`	`Option<String>`	`None`	Display name (falls back to the id).
`agent`	`Option<String>`	`None`	Agent to run (falls back to `defaultAgent`).
`prompt`	`String`	`""`	Prompt template; `{{payload}}` is replaced with the trigger payload.
`trigger`	`AutomationTrigger`	required	`{ "kind": "webhook" }` or `{ "kind": "schedule", "schedule": "every 2m" }`.
`backend`	`Option<String>`	`None` (`local`)	Execution backend the run is confined to (FR-033).
`dispatch`	`Vec<DispatchTarget>`	`[]`	Third-party notifications on run completion.
`enabled`	`bool`	`true`	Whether the automation is active.

Each dispatch target: `kind` (`slack`/`github`/`linear`/`notion`/`webhook`), optional `url`, optional `token_env` (the environment variable holding the bearer token - the value is never stored, FR-035), and optional `target` (a channel, repository, or database id). A target with no `url` is inert.

See `docs/howtos/slashcommands/automation.md` for the surfaces.

7.40 `tools`

Tool-name alias resolution, alias advertisement and the provider payload trace (spec `uniformtools` FR-015, FR-017). Every switch is off by default and the whole section is omitted from a serialised config while it is at those defaults, so a build that has never seen the section writes a byte-identical `ragent.json` and resolves every inbound tool name against the registry alone.

```
{
  "tools": {
    "aliases": { "WebFetch": "webfetch" },
    "expose_alias_names": true,
    "wire_trace": false
  }
}
```

Field	Type	Default	Description
`aliases`	`bool` or `object`	absent (off)	Provider tool spellings resolved onto ragent tool names before the unknown-tool check (FR-015). `true` requests the built-in table of provider spellings; an object adds explicit `"<provider spelling>": "<ragent tool>"` entries. Both may be in force at once after a merge.
`expose_alias_names`	`bool`	`false`	Additionally advertise each alias spelling in the outbound tool list, so a model that only knows the provider spelling can still see it (FR-015). Never advertises a retired ragent spelling (FR-031).
`wire_trace`	`bool`	`false`	Write each step's request body, redacted through the existing sanitizer, to `target/temp/wire-trace/<session>-<step>.json` (FR-017). Inert while absent or `false`, and it never touches a provider or a tool.

Four values, four meanings for `aliases`. The key is deliberately not a plain `Option<bool>`, because absent, `true`, `false` and an object (possibly empty) each say something different:

Value	Meaning
absent	This layer has no opinion; whatever a lower layer contributed stands.
`true`	Use the built-in provider-spelling table (`WebFetch` -> `webfetch`, `WebSearch` -> `websearch`, `file_search` -> `codeindex_search`, `web_search` -> `websearch`).

Value	Meaning
`false`	Switch alias resolution off for the merged config and drop every inherited entry.
`{ ... }`	Contribute these entries. `{}` contributes no spelling and clears nothing - it is not the `false` switch and round-trips as `{}`.

Layered merge. Entries union per spelling with the higher-precedence layer winning a collision, so a project file can add or re-point one spelling without discarding the user-global table. The two opt-in booleans use OR semantics: a layer that enables one cannot have it silently turned off by a layer that leaves it at the default. A project-local `.ragent/ragent.json` may contribute the section like any other non-privileged key, because it resolves *names*, not capabilities - the permission layer, the loop restriction and the 7-layer bash validation still run on the resolved call.

A retired spelling is refused when the config loads (FR-031). `tools.aliases` is the one section that can carry a tool name, so a source key or a target value equal to a before-rename spelling - including the self-mapping form - stops the load with an error naming the file, the offending spelling and its replacement. A target naming no *currently registered* tool is not refused at load (a plugin, MCP or connector tool may be bridged later); it is reported once and refused when used.

Errors are load errors, not silent defaults. A bare string, or an entry whose value is not a string, fails the load naming the accepted shapes and the offending spelling - a typo must not quietly disable alias resolution.

See section 3 for the merge-strategy table, and `QUICKSTART.md` for the resolver's inbound behaviour.

7.41 `agents`

Sub-agent panel and launch configuration (spec `agentssetup` FR-021, FR-036, NFR-004). Both keys are omitted from a serialised config while they are at their defaults, so a `ragent.json` that has never set them round-trips byte-identically.

```
{
  "agents": {
    "completed_history": 20,
    "max_launching": 4
  }
}
```

Field	Type	Default	Description
`completed_history`	`usize`	`20`	Cap on the Agents panel's completed-history section (`DEFAULT_AGENTS_COMPLETED_HISTORY`). Clamped `1..=200`.

Field	Type	Default	Description
`max_launching`	`usize`	`4`	Cap on concurrently launching sub-agent tasks across every TUI launch path (`DEFAULT_AGENTS_MAX_LAUNCHING`). Clamped `1..=64`.

Why `completed_history` exists. The panel's completed section keeps finished tasks visible newest-first after their live row is gone, which is how a `[memory]` report and a settled verdict stay readable. Unbounded, that section would grow with the number of tasks the session ever spawned, so it is capped: the panel's memory cost stays constant in session length.

Why `max_launching` exists. `/spawn` used to hold its own single-slot `Mutex`, so only one launch could be in flight from that command while every other path behaved differently. Every TUI launch now registers on one shared, bounded launch set: a second launch queues behind the busy ones and names them rather than being refused outright.

Zero is clamped up, not honoured. An explicit `0` for either key is raised to `1` rather than accepted: zero would silently switch the feature off while still claiming to be configured, which is worse than a bound the operator can see. An out-of-range value is clamped to the nearest bound.

7.42 `storage`

Session-message retention. **Opt-in**: with no `storage` section no message is ever expired, and the section is omitted from a saved file while at its default, so an untouched `ragent.json` round-trips byte-identically.

```jsonc
{
  "storage": {
    "retention": { "enabled": true, "message_days": 14 }
  }
}
```

Field	Type	Default	Description
`retention.enabled`	`bool`	`false`	Run age-based message expiry at startup and permit `/db prune` / `ragent db prune`.
`retention.message_days`	`u32`	`14`	Days a message is kept. Must be `>= 1`; a zero-day window is rejected because it would empty the store on every start.

Sessions are kept. The prune deletes from `messages_fts`, `message_fts_keys` and `messages` in one transaction and never touches `sessions`, so an expired conversation stays listed by `ragent session list` and can be resumed - it just reads back empty.

The cutoff is fallible. `chrono::Duration::checked_sub_signed` (SEC-049) means an extreme window returns an error instead of panicking inside the startup path, and `Storage::count_messages_older_than` applies the same predicate for `--dry-run`, so a report and the prune that follows it cannot disagree.

The file does not shrink. `DELETE` returns pages to SQLite's freelist; `ragent.db` keeps its size until a `VACUUM`, which needs exclusive access - stop every other ragent process first. See `docs/sqlite.md` for the storage-level detail.

Surfaces: the startup pass (on a blocking thread, so it never stalls startup), `ragent db prune [--days N] [--dry-run]`, and the TUI `/db prune [--days N] [--dry-run]` (confirmed, with `No` preselected).

---

8. Full Example File

The following is a comprehensive example showing every section. You do not need all of these — every section has defaults, so an empty `{}` is valid.

```jsonc
{
  "username": "alice",
  "defaultAgent": "coder",

  "provider": {
    "anthropic": {
      "env": ["ANTHROPIC_API_KEY"],
      "thinking": { "enabled": true, "level": "low" },
      "models": {
        "claude-sonnet-4-20250514": {
          "thinking": {
            "enabled": true,
            "level": "high",
            "budget_tokens": 16000
          }
        }
      }
    },
    "openai": {
      "env": ["OPENAI_API_KEY"]
    },
    "generic_openai": {
      "env": ["GENERIC_OPENAI_API_KEY"],
      "api": { "base_url": "http://127.0.0.1:8080" }
    },
    "router": {
      "enabled": true,
      "tiers": {
        "SIMPLE": {
          "models": [{ "provider": "ollama", "model": "llama3.2" }]
        },
        "MEDIUM": {
          "models": [
            { "provider": "anthropic", "model": "claude-sonnet-4-20250514" }
          ]
        },
```

```
      "COMPLEX": {
        "models": [
          { "provider": "anthropic", "model": "claude-sonnet-4-20250514" }
        ]
      },
      "REASONING": {
        "models": [{ "provider": "openai", "model": "o3" }]
      }
    }
  }
},

"agent": {
  "coder": {
    "model": "anthropic/claude-sonnet-4-20250514",
    "temperature": 0.2,
    "max_steps": 200,
    "skills": ["rust-review"]
  }
},

"permission": [
  { "permission": "file:write", "pattern": "src/**", "action": "allow" },
  { "permission": "file:write", "pattern": "secrets/**", "action": "deny" },
  { "permission": "bash", "pattern": "git push --force", "action": "deny" }
],

"command": {
  "build": {
    "command": "cargo build --release",
    "description": "Build release binary"
  }
},    "mcp": {
    "filesystem": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-filesystem", "/home/user/docs"]
    }
  },

  "instructions": [
    "Always use rustfmt before writing Rust files."
  ],

  "skill_dirs": ["~/my-skills"],

  "experimental": {
    "parallel_tool_calls": false,
    "max_background_agents": 8,
```

```
  "background_agent_timeout": 3600
},

"bash": {
  "allowlist": ["curl", "wget"],
  "denylist": ["git push --force"]
},

"dirs": {
  "allowlist": ["src/**", "tests/**"],
  "denylist": ["secrets/**", ".env"]
},

"tools": {
  "aliases": { "WebFetch": "webfetch" },
  "expose_alias_names": true,
  "wire_trace": false
},

"code_index": {
  "enabled": true,
  "max_file_size": 1048576,
  "extra_exclude_dirs": ["vendor"]
},

"stream": {
  "initial_response_timeout_secs": 300,
  "timeout_secs": 120,
  "max_retries": 4,
  "retry_backoff_secs": 2
},

"memory": {
  "enabled": true,
  "structured": { "enabled": true },
  "retrieval": {
    "max_memories_per_prompt": 100,
    "max_memory_tokens": 4000,
    "recency_weight": 0.3,
    "relevance_weight": 0.7
  },
  "auto_extract": {
    "enabled": false,
    "require_confirmation": true
  },
  "decay": {
    "factor": 0.95,
    "min_confidence": 0.1
  },
```

```
    "cross_project": {
      "enabled": false,
      "search_global": true,
      "project_override": true
    }
  },

"compaction": {
  "auto": true,
  "threshold": 0.7,
  "buffer": 0.10,
  "keep": { "tokens": 0.20 }
},

"gitlab": {
  "instance_url": "https://gitlab.example.com",
  "token": "env:GITLAB_TOKEN",
  "username": "myuser"
},

"yolo": false,
"edit_log": false,

"prices": [
  {
    "model": "gpt-4o",
    "input_per_1m": 2.50,
    "output_per_1m": 10.00
  }
],
"telemetry": {
  "otel": {
    "enabled": false,
    "endpoint": "http://localhost:4318",
    "protocol": "http",
    "export_interval_seconds": 30,
    "export_timeout_seconds": 10,
    "service_name": "ragent",
    "cardinality_limit": 1000,
    "logs": { "enabled": false, "level": "info" }
  }
},

"agent_perf": {
  "enabled": true,
  "step_budget_secs": 300,
  "stall_timeout_secs": 60,
  "max_concurrent_tools": 4,
  "parallel_independent_tools": true
```

```
},
"tavily_api_key": null,
"langsearch_api_key": null,
"perplexity_api_key": null,
"exa_api_key": null,
"serper_api_key": null,
"openalex_email": null,

"sdd": {
  "clarification_markers": false,
  "quality_checklists": false
},

"trigger": {
  "enabled": true,
  "poll_interval_secs": 30,
  "max_rules": 32
},

"piegap": {
  "triggers": false,
  "hooks": false,
  "undo": false
},

"research": {
  "open_access_recovery": false,
  "exclude_academic_engines": false,
  "contact_email": null,
  "oa_min_full_text_chars": 1000,
  "evaluate": {
    "enabled": false
  }
},

"loop": {
  "max_steps": 512,
  "cost_limit": null,
  "error_retry_allowance": 3,
  "checkpoints": true,
  "checkpoint_timeout_secs": 120
},

"activity_log": true,

"plugins": {
  "enabled": true,
  "max_execution_ms": 5000,
  "max_entry_ms": 10000,
```

```
    "max_memory_mb": 64,
    "store_dir": null,
    "permissions": {},
    "stores": {
      "codex":  { "url": null },
      "claude": { "url": null },
      "marketplaces": [],
      "timeout_ms": 10000,
      "max_index_bytes": 2097152,
      "cache_ttl_secs": 3600
    }
  },

  "connectors": {
    "enabled": true,
    "store_dir": null,
    "stores": {
      "timeout_ms": 10000,
      "max_index_bytes": 2097152,
      "cache_ttl_secs": 3600,
      "marketplaces": []
    },
    "credentials": {}
  },

  "input_queue_capacity": 32,

  "acp": {
    "default_agent": "claude",
    "server_enabled": false,
    "server_agent": null,
    "agents": {
      "claude": { "id": "claude", "command": "claude-code-acp", "args": [] }
    }
  },

  "storage": {
    "retention": { "enabled": true, "message_days": 14 }
  }
}
```

---

9. Common Recipes

Minimal config (just an API key reference)

```
{
  "provider": {
    "anthropic": { "env": ["ANTHROPIC_API_KEY"] }
```

```
  }
}
```

The API key itself is read from the environment variable; it does not need to be in the config file.

Local-first (Ollama only)

```
{
  "defaultAgent": "coder",
  "provider": {
    "ollama": {}
  }
}
```

ragent resolves to the first available local/self-hosted provider when no model is explicitly configured.

Custom OpenAI-compatible endpoint

```
{
  "provider": {
    "generic_openai": {
      "env": ["GENERIC_OPENAI_API_KEY"],
      "api": { "base_url": "http://127.0.0.1:8080" }
    }
  }
}
```

The base URL can also be set via `GENERIC_OPENAI_API_BASE` env var.

Allow writes to src/, deny secrets

```
{
  "permission": [
    { "permission": "file:write", "pattern": "src/**", "action": "allow" },
    { "permission": "file:write", "pattern": "secrets/**", "action": "deny" }
  ]
}
```

Disable code indexing

```
{
  "code_index": { "enabled": false }
}
```

OpenTelemetry export to a collector

```
{
  "telemetry": {
    "otel": {
      "enabled": true,
      "endpoint": "http://otel-collector:4318",
      "protocol": "http",
```

```
      "export_interval_seconds": 15,
      "service_name": "ragent-prod",
      "resource_attributes": {
        "deployment.environment": "production",
        "host.name": "build-server-01"
      }
    }
  }
}
```

Model Router with fallback chains

```
{
  "provider": {
    "router": {
      "enabled": true,
      "tiers": {
        "SIMPLE": {
          "models": [
            { "provider": "ollama", "model": "llama3.2" },
            { "provider": "openai", "model": "gpt-4o-mini" }
          ]
        },
        "MEDIUM": {
          "models": [
            { "provider": "anthropic", "model": "claude-sonnet-4-20250514" },
            { "provider": "openai", "model": "gpt-4o" }
          ]
        },
        "COMPLEX": {
          "models": [
            { "provider": "anthropic", "model": "claude-sonnet-4-20250514" }
          ]
        },
        "REASONING": {
          "models": [
            { "provider": "openai", "model": "o3" },
            { "provider": "anthropic", "model": "claude-sonnet-4-20250514" }
          ]
        }
      }
    }
  }
}
```

Run a session inside a container sandbox

```
{
  // Run shell/file/search tools inside a Podman sandbox. The workspace is a
```

```
  // container-named volume (never a host bind mount), so the host working
  // directory is untouched by sandboxed tools.
  "execution_backend": "podman",
  "backends": [
    { "id": "sandbox", "kind": "podman",
      "image": "ghcr.io/yourorg/ragent-sandbox:latest" }
  ]
}
```

With no runtime specified the backend defaults to `podman` (FR-026); substitute `"docker"` when only Docker is present. If the runtime is missing or the image cannot be pulled, the turn fails with the provisioning error and the tool is not run on the host (FR-031).

Override pricing for a custom model

```
{
  "prices": [
    {
      "model": "my-custom-model",
      "input_per_1m": 1.00,
      "output_per_1m": 5.00
    }
  ]
}
```

Inline config via environment variable

```
RAGENT_CONFIG_CONTENT='{"defaultAgent":"ask","yolo":true}' ragent run "hello"
```

This is merged last, with the highest precedence.

---

10. Related Documents

Document	Topic
`docs/howtos/tutorial.md`	End-to-end tutorial from setup to release
`docs/howtos/permissions.md`	Permissions, autopilot, and YOLO mode
`docs/howtos/tools/INDEX.md`	Complete tool reference (split by category)
`docs/howtos/custom-agents.md`	Custom agent definitions (OASF / Markdown)
`docs/howtos/teams.md`	Teams and swarm coordination
`docs/howtos/hooks.md`	Lifecycle hooks
`docs/howtos/llmsecurity.md`	LLM security analyzer (tightening-only layer)
`docs/howtos/sandbox-backends.md`	Switchable execution backends and container sandboxing

Document	Topic
docs/howtos/acp.md	Agent Client Protocol (ACP) client and server
docs/howtos/codeindex.md	Code index (tree-sitter, search, graph)
docs/howtos/spec.md	Spec lifecycle management
docs/howtos/research.md	Research system
docs/howtos/reverse.md	GitHub repo reverse-engineering
README.md	Project overview and quick start
SPEC.md	Full configuration schema reference
QUICKSTART.md	Quick start guide